WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

lottie的设计与实现

主讲人：李老师

lottie介绍和特性

lottie的设计与实现

效果演示

互动交流

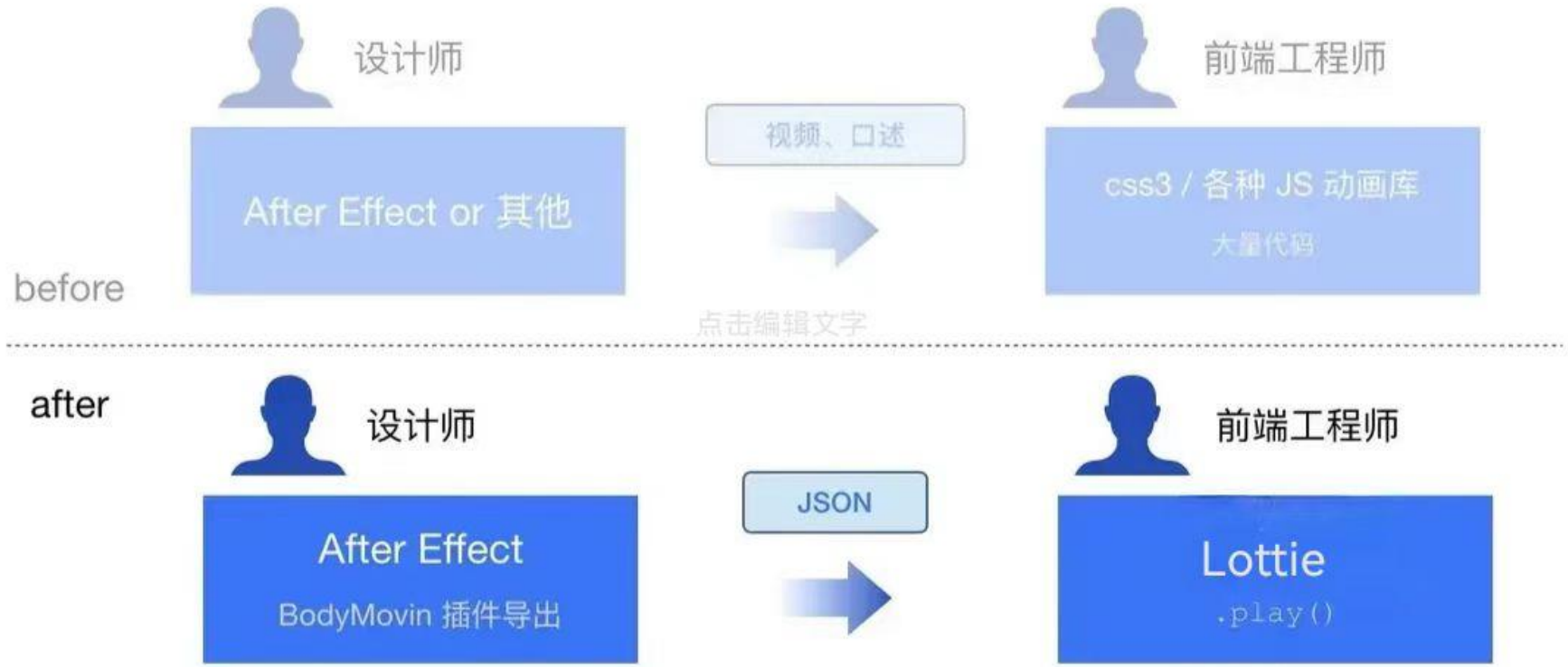


lottie的介绍

- Lottie的介绍
- Lottie的核心优势
- Lottie的使用
- Lottie的应用场景

什么是 Lottie4cj?

lottie4cj是一个动画库，设计师可以使用 Adobe After Effects 设计出漂亮的动画之后，使用Adobe After Effects软件通过Bodymovin插件导出成 JSON 格式，就可以在移动设备上进行本地渲染。



Lottie 的核心优势

易于使用

高性能

高灵活性

视觉效果精美

动画设计和动画播放分离

开发者无需编写复杂的动画代码，只需加载和播放 JSON 文件即可。

基于JSON格式

JSON是一种轻量级的数据交换格式，体积小，加载速度快，具有良好的可读性和解析性能。

支持多种操作

前端可以控制动画的播放、暂停、循环、速度调整，甚至播放特定的动画片段。

渲染流畅

Lottie 支持 Canvas 渲染，展示效果细腻，可以精准还原设计师在 After Effects 中的创意动画。

仓颉 Cangjie

如何使用 Lottie

1. 设计师：

- 使用 After Effects 制作动画。
- 通过 Bodymovin 插件导出 JSON 文件。

2. 开发者：

- 集成 Lottie。
- 加载并播放 JSON 动画文件。

代码示例：

```
this.animateItem = lottie.loadAnimation(AnimationConfig(
    container: this.mainCanvasRenderingContext,
    renderer: "canvas",
    loop: -1,                    // 是否循环播放,默认 -1 循环。
    autoplay: true,              // 是否自动播放，默认true
    name: "2016",                // 动画名称
    contentMode: "Contain", // 填充的模式
    path: "data.json",
    context: globalAbilityContext
))
```

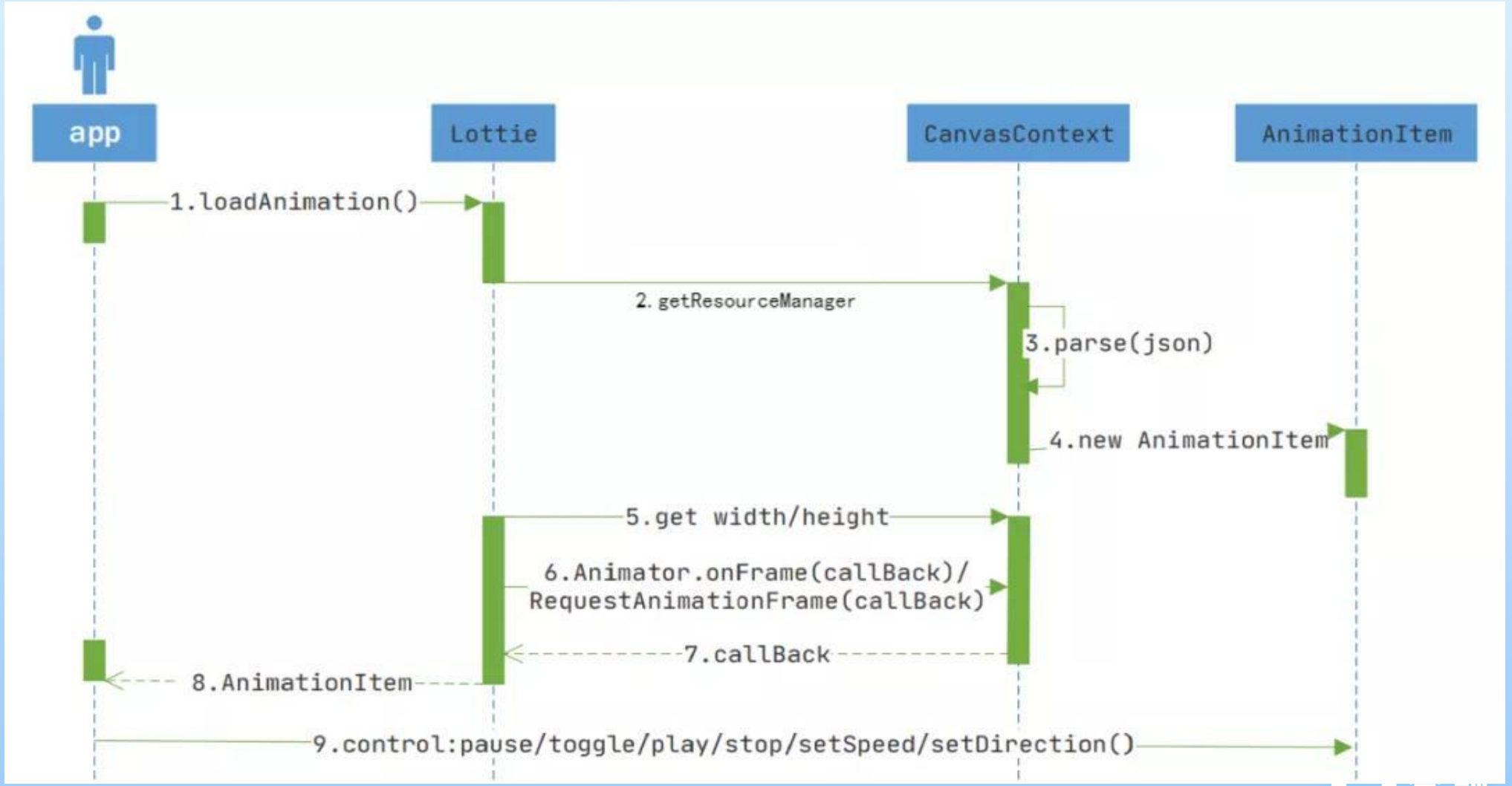
Lottie 的应用场景

- UI 动画：按钮点击、加载动画、转场效果。
- 品牌宣传：动态 Logo、品牌故事动画。
- 教育领域：交互式教学动画。
- 游戏：轻量级游戏动画。
- 广告：动态广告横幅。

lottie的设计与实现

- Lottie绘制流程
- 实现的特性
- 测试用例
- json动画数据格式

Lottie绘制流程图：



仓颉 Cangjie

Lottie绘制流程：

Lottie解析json格式的动画描述文件后，会基于canvas 画布进行2D渲染，并结合原生组件Animator实现动画效果。

1. 动画加载准备，在使用Lottie加载动画前需先通过插件 bodymovin 将AE生成的动画文件转换为通用的 json 格式描述文件。开发者也可以从互联网获取合适的动画资源直接应用。
2. 获取json文件中的动画数据。
3. 解析json文件中的动画数据。
4. 创建动画实例，设置动画信息。
5. 初始化布局宽高，设置绘制样式等信息。
6. 启动动画，触发逐帧绘制。
7. 更新动画进度。
8. 返回动画实例，通过loadAnimation()接口返回动画实例AnimationItem。
9. 控制动画，Lottie提供了一整套简洁易用API，如停止stop()、暂停pause()、播放play()、播放流转togglePause()、方向setDirection()、速度setSpeed()等。

实现的特性：

- 支持加载本地文件及raw文件夹下文件方式加载动画
- 支持加载网络资源以URI路径方式加载动画,支持json和zip/gzip格式
- 支持构建animationData对象加载动画
- 支持动画播放、暂停、销毁等功能

构建animationData对象加载动画

```
let path: String = "data.json"
let animationData: AnimationData = AnimationDataParser.parse(String.fromUtf8(loadMedia(path,globalAbilityContext)))
```

```
lottie.destroy(animationName: "2016") //加载动画前先销毁之前加载的动画
this.lottie: AnimationManager = AnimationManager(this.mainCanvasRenderingContext)
this.animateItem = lottie.loadAnimation(AnimationConfig(
    container: this.mainCanvasRenderingContext,              // 渲染上下文
    renderer: "canvas",                                      // canvas 渲染模式
    loop: -1,                                                // 是否循环播放,默认 -1 循环。
    autoplay: true,                                          // 是否自动播放，默认true
    name: "2016",                                            // 动画名称
    contentMode: "Contain",                                  // 填充的模式
    animationData: animationData,                            // json动画解析的对象
    context: globalAbilityContext
))
或
this.animateItem = lottie.loadAnimation(AnimationConfig(
    container: this.mainCanvasRenderingContext,        // 渲染上下文
    renderer: "canvas",                                // canvas 渲染模式
    loop: -1,                                          // 是否循环播放,默认 -1 循环。
    autoplay: true,                                    // 是否自动播放，默认true
    name: "2016",                                      // 动画名称
    contentMode: "Contain",                            // 填充的模式
    path: "data.json",                                 // 路径加载动画支持：1.raw文件夹下的相对路径和本地路径，2.本
    context: globalAbilityContext
))
```

加载raw文件夹下json文件方式加载动画

加载本地文件

```
this.animateItem = lottie.loadAnimation(AnimationConfig(
    container: this.mainCanvasRenderingContext,              // 渲染上下文
    renderer: "canvas",                                      // canvas 渲染模式
    loop: 1,                                                 // 是否循环播放,默认 -1 循环。
    autoplay: true,                                          // 是否自动播放，默认true
    name: "2016",                                            // 动画名称
    contentMode: "Contain",                                  // 填充的模式
    path: "/data/storage/el2/base/files/entry/data.json",    // 路径加载动画支持：1.raw文件夹下的相对路径和本地路径，2.
    pathSourceFrom: 2,                                       // 区别json文件来源：1.来自\resources\rawfile\， 2.来自
    context: globalAbilityContext
))
```

或

```
this.animateItem = lottie.loadAnimation(AnimationConfig(
    uri: "https://assets7.lottiefiles.com/packages/lf20_sF7uci.json",  //来自网络路径的动画数据，支持json和zip/gzip格
    isNetwork: true,                                                   //true优先读取网络资源,false优先读取本地缓存资
    container: this.mainCanvasRenderingContext,                        // 渲染上下文
    renderer: "canvas",                                                // canvas 渲染模式
    autoplay: true,                                                    // 是否自动播放，默认true
    name: "2016",                                                      // 动画名称
    contentMode: "Contain",                                            // 填充的模式
    context: globalAbilityContext
))
```

支持加载网络资源以URI路径方式加载动画,支持json和zip/gzip格式

支持动画播放、暂停、销毁等功能

使用方法	类型	相关描述
play	name!: String = ""	播放
stop	name!: String = ""	停止
pause	name!: String = ""	暂停
togglePause	name!: String = ""	切换暂停
destroy	name!: String = ""	销毁动画
goToAndStop	value: Float64, isFrame!: Bool = false, name!: String = ""	控制动画画面停止在某一帧或某个时刻,判断按帧还是按毫秒控制，true 按帧控制，false 按时间控制
goToAndPlay	value: Float64, isFrame!: Bool = false, name!: String = ""	控制动画从某一帧或某一时刻开始播放,判断按帧还是按毫秒控制，true 按帧控制，false 按时间控制
setSegment	initNum: Float64, end: Float64	设置动画片段
playSegments	arr: Array<Array<Float64>>, forceFlag: Bool	播放指定片段
resetSegments	forceFlag: Bool	重置动画
setSpeed	val: Float64	设置播放速度
setDirection	val: Int64	设置播放方向
getDuration	isFrame: Bool	获取动画时长
addEventListener	eventName: String, callback: AnimationEventCallback	添加监听状态
removeEventListener	eventName: String, callback: AnimationEventCallback, hascallback!: Bool = true	移除监听状态
changeColor	color: Array<Int64>, layer: Int64, index: Int64	更改不含图片json的动画颜色
setContentMode	contentMode: String	设置填充模式

json动画数据格式

```
{
    "v": "5.4.2",
    "fr": 30,
    "ip": 0,
    "op": 180,
    "w": 750,
    "h": 1334,
    "nm": "动画",
    "ddd": 0,
    "assets": [ ⋯
    ],
    "layers": [ ⋯
    ],
    "markers": [ ]
}
```

fr：帧率（fps）
ip：开始帧
op：结束帧
w ：场景宽
h：场景高

assets：资源信息
layers：动画层级

```
{
 "layers": [{
     "ddd": 0,
     "ind": 0,
     "ty": 4,
     "nm": "形状图层 1",
     "ks": {
         "o": {/* ... */},
         "r": {/* ... */},
         "p": {/* ... */},
         "a": {/* ... */},
         "s": {/* ... */},
     },
     "ao": 0,
     "shapes": [/* ... */],
     "ip": 0,
     "op": 180,
     "st": 0,
     "bm": 0,
     "sr": 1
     }]
}
```

ddd：是否为3d
ind：图层id
ty: 图层类型
nm: 图层名字

ks: 图层对应变化属性
o：opacity 透明度
r：rotation 旋转
p：position 位置
a：anchor 锚点
s：scale 缩放

shaps：图层宽高等

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，获取最新官方资讯!
(您也可以替换为自己的二维码/联系方式，邀请社区开发者共建项目)

